

Apunte Semana 14 Clase II Inteligencia Artificial

Fabrizio Picado Alvarado
2022104933
Escuela de Ingeniería en Computación
Tecnológico de Costa Rica
Profesor: Steven Pacheco Portuguez
Curso: IC6200 Inteligencia Artificial
Fecha: 28 de mayo de 2026

I. REPASO DE LA CLASE ANTERIOR

I-A. Concepto general de agente LLM

sistema basado en un modelo de lenguaje que no se limita a producir respuestas textuales. Su función puede incluir el uso de información previa, la consulta de herramientas externas y la organización de acciones para resolver una tarea.

De esta manera, el agente funciona como una extensión del modelo de lenguaje, ya que puede interactuar con otros recursos y no depender únicamente del conocimiento interno del modelo.

I-B. Componentes principales de un agente

estos agentes requieren componentes adicionales al modelo de lenguaje. Los principales son la memoria, las herramientas y la planificación. La memoria permite conservar información relevante; las herramientas permiten consultar o utilizar servicios externos; y la planificación organiza los pasos necesarios para cumplir un objetivo.

Estos elementos permiten que el sistema pase de una respuesta conversacional simple a un proceso más autónomo, estructurado y orientado a tareas.

I-B1. Memoria: permite que el agente conserve información de interacciones anteriores, como preferencias del usuario, datos relevantes o información previamente mencionada. Esto evita que cada interacción sea tratada como completamente nueva. Gracias a esto el agente puede mantener continuidad entre sesiones, adaptar sus respuestas al contexto acumulado y responder de forma más coherente y útil.

I-B2. Herramientas: permiten que un agente basado en LLM consulte servicios externos o ejecute acciones específicas, en lugar de depender únicamente del conocimiento interno del modelo, ampliando sus capacidades y le permite trabajar con información actualizada.

Por ejemplo, un agente orientado a planificar un viaje podría consultar APIs de vuelos, clima, calendarios y hospedaje. Así, no solo genera recomendaciones generales, sino que puede apoyarse en datos externos para construir una respuesta más útil y completa.

I-B3. Planificación: permite que el agente divida una tarea en pasos y los ejecute de forma ordenada. Esto es necesario porque muchas tareas no se resuelven con una sola respuesta, sino que requieren identificar información, consultar herramientas, analizar resultados y producir una solución.

para el ejemplo del agente de viaje, para organizar un viaje, podría identificar destino y fechas, revisar vuelos, consultar clima, buscar hospedaje y finalmente presentar una propuesta. La planificación permite que el sistema actúe de manera más estructurada y cercana a un agente autónomo.

I-C. Tipos de memoria

puede dividirse en memoria de corto plazo y memoria de largo plazo. Ambas permiten mantener coherencia y continuidad, pero se diferencian en el tiempo durante el cual conservan la información.

La memoria de corto plazo está relacionada con la interacción actual, mientras que la memoria de largo plazo conserva información persistente para futuras sesiones.

I-C1. Memoria de corto plazo: Corresponde a la información disponible dentro de la ventana de contexto del modelo. Incluye los mensajes recientes, instrucciones, datos proporcionados por el usuario e información utilizada durante la tarea actual.

Esta memoria es temporal y permite mantener coherencia dentro de una misma conversación o proceso. Sin embargo, no se conserva necesariamente después de finalizar la interacción, por lo que no es suficiente cuando se necesita recordar información entre sesiones diferentes.

I-C2. Memoria de largo plazo: La memoria de largo plazo permite conservar información persistente entre diferentes interacciones o sesiones. A diferencia de la memoria de corto plazo, no depende únicamente de la ventana de contexto actual, sino que permite recuperar datos relevantes después de que una conversación haya terminado.

Puede almacenar preferencias del usuario, datos históricos o información útil para futuras tareas. Por lo que el agente no debe empezar desde cero en cada interacción, pero es necesario definir qué información se guarda, durante cuánto tiempo y bajo qué condiciones, ya que no todos los datos son permanentes o relevantes.

II. MATERIA DESARROLLADA

No todos los modelos sirven para los mismos propósitos, por lo que la elección depende del tipo de tarea, las restricciones del sistema y las capacidades requeridas.

II-A. Selección de un modelo de lenguaje

Al construir un agente, no basta con utilizar cualquier LLM. Es necesario seleccionar el modelo según las necesidades del sistema y la tarea que se desea resolver.

Cada modelo puede diferenciarse en precisión, costo, velocidad, capacidad de razonamiento, ventana de contexto, privacidad y soporte para herramientas. Por ello, el modelo no debe elegirse solo por ser el más avanzado, sino por su adecuación al contexto de uso.

En tareas simples puede ser suficiente un modelo más rápido y económico, mientras que tareas complejas pueden requerir modelos más avanzados o especializados. En agentes, esta decisión es clave porque el modelo puede encargarse de interpretar instrucciones, usar herramientas, mantener contexto y planificar acciones.

II-B. Factores para seleccionar un modelo

Depende de las necesidades de la aplicación y de las restricciones del entorno donde se utilizará el agente. Entre los factores principales se encuentran la precisión requerida, el tiempo de respuesta, el costo computacional, la privacidad de los datos, la ventana de contexto, la capacidad de razonamiento y el soporte para herramientas externas. Por lo que la elección debe buscar un equilibrio entre rendimiento, costo, velocidad y control.

II-B1. Precisión requerida: La precisión requerida corresponde al nivel de exactitud que necesita la tarea. No todas las aplicaciones requieren el mismo grado de confiabilidad, por lo que el modelo debe elegirse según la importancia de producir respuestas correctas.

Tareas simples, como responder preguntas frecuentes o clasificar información general, pueden funcionar con modelos menos complejos. En cambio, tareas que requieren análisis cuidadoso, razonamiento detallado o decisiones más delicadas necesitan modelos con mayor precisión.

II-B2. Tiempo de respuesta: Se relaciona con la latencia, es decir, el tiempo que tarda el modelo en generar una respuesta. Algunos modelos más complejos requieren más procesamiento, por lo que pueden ser más lentos.

En aplicaciones que necesitan interacción rápida, como asistentes simples o sistemas de preguntas frecuentes, conviene utilizar modelos más livianos. En cambio, si la tarea requiere análisis profundo, puede aceptarse una mayor latencia.

II-B3. Costo computacional: Recursos necesarios para ejecutar un modelo, como procesamiento, memoria e infraestructura. Los modelos más grandes o avanzados suelen requerir más recursos y generar mayores costos.

Se debe evaluar si realmente se necesita un modelo grande. Para tareas simples, un modelo avanzado puede ser innecesario.

II-B4. Privacidad de los datos: La privacidad de los datos es importante cuando el agente trabaja con información sensible, datos internos o información que no debería enviarse a servicios externos sin control.

En estos casos, se debe decidir entre utilizar un modelo propietario ofrecido por un proveedor externo o un modelo

abierto desplegado en infraestructura propia. Los modelos propietarios suelen facilitar la integración, pero implican dependencia del proveedor. Los modelos abiertos ofrecen mayor control sobre los datos, aunque requieren más trabajo técnico para desplegarlos y administrarlos.

II-B5. Ventana de contexto: La ventana de contexto es la cantidad de información que el modelo puede recibir y manejar dentro de una misma interacción. Una ventana más amplia permite trabajar con documentos largos, conversaciones extensas o tareas que requieren mantener varios datos presentes.

En agentes, esto es útil para analizar instrucciones largas o información acumulada durante una tarea. Sin embargo, y no debe confundirse con la memoria persistente: la información en la ventana de contexto es temporal y no necesariamente se conserva para futuras sesiones.

II-B6. Capacidad de razonamiento: La capacidad de razonamiento indica qué tan bien un modelo puede analizar una tarea, dividirla en pasos y construir una respuesta elaborada. No todos los modelos razonan con la misma profundidad, por lo que este factor debe considerarse según la complejidad del problema.

Tareas de programación, resolución de problemas complejos, planificación o análisis detallado pueden necesitar modelos con mayor capacidad de razonamiento.

II-B7. Soporte para tools: permite que el modelo se conecte con sistemas externos, consulte servicios o ejecute acciones. Es especialmente importante en agentes, ya que no basta con generar texto, sino que puede ser necesario interactuar con APIs, calendarios, clima u otras fuentes de información.

II-C. Modelos propietarios y abiertos

Los modelos propietarios suelen ofrecer integración sencilla, infraestructura administrada, buen rendimiento y soporte para herramientas. Son convenientes cuando se busca desarrollar una solución rápidamente, aunque implican dependencia del proveedor, costos por uso y menor control sobre el modelo y los datos.

Los modelos abiertos ofrecen mayor control, privacidad, personalización y posibilidad de despliegue local o en infraestructura propia. Sin embargo, requieren más complejidad operativa, recursos computacionales, mantenimiento y monitoreo.

II-D. Modelos de razonamiento

Se utilizan cuando una tarea requiere análisis profundo, pasos intermedios o resolución compleja. A diferencia de modelos orientados a respuestas directas, dedican más procesamiento a interpretar el problema y construir una solución.

II-D1. Características generales: Los modelos de razonamiento están diseñados para tareas que requieren análisis, planificación o resolución en varios pasos. Son útiles cuando la respuesta no depende solo de recuperar información, sino de evaluar posibilidades, seguir instrucciones complejas, resolver problemas técnicos o coordinar acciones.

II-D2. Cuándo utilizar modelos de razonamiento: Los modelos de razonamiento son adecuados cuando la tarea requiere análisis detallado, varios pasos o resolución de problemas complejos. Son útiles en matemática, programación, planificación, búsqueda de información, análisis multipaso y coordinación de herramientas externas.

En programación pueden ayudar a revisar errores, explicar código o proponer soluciones. En planificación, permiten organizar acciones considerando restricciones y condiciones. En estos casos, el modelo no solo genera una respuesta, sino que sigue un proceso más estructurado para llegar a una solución.

II-D3. Cuándo no utilizar modelos de razonamiento: Para tareas simples puede ser mejor utilizar modelos más rápidos, económicos y con menor latencia, ya que un modelo de razonamiento podría consumir más tiempo y recursos sin aportar una mejora significativa.

Ejemplos de tareas donde no suele ser necesario un modelo de razonamiento son responder preguntas frecuentes, clasificar texto, resumir información sencilla, traducir frases o mantener conversaciones simples.

II-E. Modelo adecuado para agentes

Un modelo de lenguaje por sí solo no necesariamente es un agente. Para convertirse en agente debe integrarse con componentes como memoria, tools, planificación, estado, restricciones y ciclos de ejecución.

La memoria conserva información relevante; las tools permiten consultar servicios externos o ejecutar acciones; la planificación divide la tarea en pasos; el estado indica el punto actual del proceso; y las restricciones controlan el comportamiento del sistema.

Por esta razón, al seleccionar un modelo para agentes no basta con evaluar la calidad de sus respuestas. También se debe considerar si puede manejar contexto, usar herramientas, seguir instrucciones, producir salidas estructuradas y participar en ciclos de ejecución.

II-F. Características importantes en agentes

Un agente necesita más que buenas respuestas textuales. Debe poder trabajar con tareas amplias, mantener información relevante y utilizar recursos externos cuando sea necesario.

Entre sus características importantes están la comprensión de contexto largo, el uso de herramientas externas y la coordinación de recursos. Estas capacidades permiten que el agente resuelva tareas más complejas que una interacción simple de pregunta y respuesta.

II-F1. Comprensión de contexto largo: La comprensión de contexto largo permite que un agente maneje instrucciones extensas, conversaciones largas o información acumulada durante una tarea. Esto es útil cuando existen varias condiciones, restricciones o detalles que deben mantenerse presentes antes de responder o actuar.

El contexto largo no debe confundirse con la memoria persistente. El contexto permite manejar más información dentro de la interacción actual, mientras que la memoria conserva información entre sesiones.

II-F2. Uso de herramientas externas: Las herramientas externas permiten que el agente consulte información actualizada o ejecute acciones que el modelo no puede realizar por sí solo, como usar APIs, calendarios, clima o servicios de búsqueda. Esta capacidad amplía el alcance del agente y permite resolver tareas más completas.

II-F3. Coordinación de recursos: Consiste en organizar información, herramientas y acciones para cumplir un objetivo. Un agente no solo debe tener acceso a herramientas, sino saber cuándo usarlas y cómo integrar sus resultados. Coordinar estos elementos permite producir una solución coherente y estructurada.

II-F4. Tool calling y structured output: El *tool calling* permite que el modelo llame herramientas específicas para consultar información o ejecutar acciones, esto separa la generación de texto de la ejecución de acciones concretas.

El *structured output* permite que la respuesta del agente tenga un formato definido y fácil de interpretar por otros sistemas. Ambos mecanismos ayudan a que el agente trabaje de forma más controlada y confiable.

II-G. Prompting para agentes

El *prompting* en agentes no consiste solo en hacer una pregunta al modelo, sino en definir cómo debe comportarse durante una tarea. El prompt puede establecer el rol del agente, sus objetivos, restricciones, herramientas disponibles y formato de salida esperado.

De esta forma, el prompt funciona como una guía que orienta el comportamiento del sistema, reduce ambigüedad y permite que el agente trabaje de manera más controlada.

II-G1. Elementos que puede definir un prompt: Un prompt puede definir el rol del agente, los objetivos de la tarea, las restricciones, las herramientas disponibles, el formato de salida y las reglas de comportamiento.

El rol establece desde qué perspectiva actúa el agente; los objetivos indican qué debe lograr; las restricciones delimitan su comportamiento; las herramientas especifican qué recursos puede usar; y el formato de salida permite pedir respuestas organizadas, listas, tablas o estructuras específicas. Estas reglas ayudan a mantener coherencia y control durante la ejecución.

II-G2. Problemas que ayuda a reducir: Un buen prompt reduce ambigüedad, alucinaciones, uso incorrecto de herramientas y respuestas innecesarias. Al definir instrucciones claras, el agente entiende mejor qué debe hacer, cómo debe responder y cuáles son sus límites.

También ayuda a indicar cuándo debe apoyarse en memoria o herramientas externas, evitando que genere información sin fundamento o que utilice recursos cuando no corresponde. De esta manera, el agente se mantiene enfocado en el objetivo y trabaja de forma más controlada.

II-H. Context engineering

El *context engineering* consiste en decidir qué información debe entrar al contexto del agente. No basta con escribir buenos prompts; también es necesario seleccionar datos relevantes, memoria recuperada, herramientas disponibles y reglas de comportamiento.

El contexto puede incluir instrucciones, información del usuario, historial de conversación, resultados de herramientas y restricciones. Si se incluye información irrelevante, el modelo puede confundirse; si se selecciona bien, el agente trabaja de forma más ordenada, precisa y útil.

II-I. Memoria en agentes

La memoria permite que un agente conserve información previa y mantenga continuidad durante una tarea o entre distintas interacciones. Esto es útil cuando el sistema debe recordar preferencias, restricciones, estados de una tarea o información mencionada anteriormente.

Sin memoria, el agente puede perder coherencia, repetir preguntas, olvidar condiciones importantes o reiniciar procesos que ya estaban avanzados.

II-J. Ventana de contexto y memoria

La ventana de contexto y la memoria están relacionadas, pero no son lo mismo. La ventana de contexto contiene información temporal disponible durante la interacción actual, como mensajes recientes, instrucciones y datos usados en la tarea. En cambio, la memoria persistente permite guardar información para recuperarla en futuras sesiones.

Esta diferencia también se relaciona con los conceptos de *working memory* y *long-term memory*. La *working memory* corresponde a la información activa durante una tarea, mientras que la *long-term memory* conserva datos relevantes entre interacciones.

Ambas son importantes en agentes: la ventana de contexto mantiene coherencia inmediata, mientras que la memoria persistente da continuidad a largo plazo.

II-K. Políticas de olvido

No toda la información debe guardarse permanentemente. Las políticas de olvido definen qué datos se conservan, cuáles se eliminan y durante cuánto tiempo permanecen almacenados.

Estas políticas son necesarias porque algunos datos pueden volverse obsoletos, perder relevancia o afectar la privacidad y seguridad del usuario. Además, guardar demasiada información puede contaminar el contexto del agente y reducir la precisión de sus respuestas. Por ello, la memoria debe mantenerse útil, segura y actualizada.

II-L. Construcción de agentes

La construcción de agentes implica integrar un modelo de lenguaje con objetivos, memoria, herramientas, acciones y mecanismos de recuperación de información. El modelo interpreta instrucciones y genera respuestas, pero necesita otros componentes para actuar de forma organizada.

La memoria conserva información relevante; las herramientas y acciones permiten interactuar con sistemas externos; y la recuperación de información aporta datos necesarios para resolver tareas. Construir un agente requiere definir cómo se conectan estos elementos, qué límites debe respetar el sistema y cómo debe manejar la información para producir resultados coherentes y útiles.

III. TAREA ASIGNADA

La tarea asignada consiste en diseñar, implementar y evaluar un sistema multiagente capaz de responder preguntas a partir de los apuntes generados del curso. El sistema debe integrar RAG, comunicación entre agentes, memoria conversacional, herramientas externas, un servidor MCP, observabilidad e interfaz web.

III-A. Objetivo general

El objetivo principal es construir una aplicación basada en agentes, donde el modelo de lenguaje no solo responda preguntas, sino que también pueda decidir qué agente consultar, qué herramienta utilizar, cuándo recuperar información, cuándo usar memoria histórica y cómo justificar sus acciones.

El sistema debe permitir responder preguntas sobre los apuntes del curso y simular el uso de información transaccional ficticia mediante un servidor MCP conectado a una base de datos relacional.

III-B. Base documental y RAG

La base documental del sistema estará formada por los apuntes del curso. Estos documentos deben procesarse para extraer su contenido textual, limpiarlo, normalizarlo y dividirlo en fragmentos mediante una estrategia de segmentación.

Posteriormente, los fragmentos deben transformarse en *embeddings* y almacenarse en una base vectorial. Esta base será utilizada por el componente RAG para recuperar información relevante y generar respuestas fundamentadas en los documentos del curso.

Aunque el enunciado menciona la comparación de dos configuraciones RAG con distintas estrategias de segmentación, posteriormente se aclaró que este requerimiento no es obligatorio. Por lo tanto, el enfoque principal debe estar en implementar un RAG funcional, correctamente documentado y capaz de recuperar información útil desde los apuntes.

III-C. Arquitectura multiagente

El sistema debe estar formado por varios agentes especializados coordinados por un agente orquestador. Este agente recibe la pregunta del usuario, decide qué agente o herramienta utilizar, coordina el flujo de trabajo y construye la respuesta final.

Como mínimo, la arquitectura debe incluir un agente orquestador, un agente RAG, un agente de búsqueda web, un agente resumidor y un agente transaccional. Cada agente debe tener un rol definido, herramientas permitidas, entradas esperadas, salidas esperadas y restricciones de uso.

III-D. Herramientas del sistema

El sistema debe contar con herramientas especializadas para apoyar el trabajo de los agentes. Entre ellas se incluye una herramienta RAG para recuperar información desde la base vectorial, una herramienta de búsqueda web para casos permitidos, una herramienta de memoria para consultar o actualizar información conversacional y una herramienta transaccional para comunicarse con el servidor MCP.

Estas herramientas deben usarse de forma controlada, de modo que el agente no ejecute acciones innecesarias ni consulte información externa sin justificación.

III-E. Servidor MCP y base de datos ficticia

La tarea también requiere implementar un servidor MCP que permita consultar una base de datos relacional ficticia. Esta base debe ejecutarse mediante Docker y contener información simulada, como clientes, cuentas, transacciones y casos de revisión o fraude.

El servidor MCP no debe permitir consultas SQL libres. En su lugar, debe exponer herramientas controladas, por ejemplo, para buscar transacciones, consultar el resumen de riesgo de un cliente o revisar transacciones sospechosas recientes.

También se deben definir reglas de seguridad para el uso del MCP, como justificar cada llamada, evitar consultas masivas sin filtros, anonimizar información sensible y no mostrar números de cuenta completos.

III-F. Memoria conversacional

El sistema debe implementar memoria temporal de sesión y memoria histórica persistente. La memoria temporal permite mantener coherencia entre preguntas consecutivas dentro de una misma conversación.

La memoria histórica permite consultar información de sesiones anteriores, como preguntas previas, respuestas anteriores o resúmenes de conversaciones pasadas. Esto permite que el agente responda preguntas de seguimiento y mantenga continuidad entre interacciones.

III-G. Observabilidad

La ejecución del sistema debe registrarse mediante Langfuse o una herramienta equivalente. La observabilidad debe permitir revisar la entrada del usuario, el agente que recibió la solicitud, las herramientas utilizadas, los fragmentos recuperados por RAG, las llamadas al MCP, las llamadas al modelo, la latencia, los errores y, cuando sea posible, el costo aproximado.

Este registro es importante porque permite explicar qué agentes y herramientas participaron en cada consulta, por qué fueron utilizados y qué resultado produjeron.

III-H. Interfaz web y demostración

El sistema debe contar con una interfaz web que permita interactuar con el agente. Esta interfaz debe permitir demostrar preguntas sobre los apuntes, preguntas de seguimiento con memoria, búsquedas web y consultas a la base transaccional mediante MCP.

Durante la demostración también se debe evidenciar que el sistema puede rechazar o anonimizar solicitudes relacionadas con información sensible.

III-I. Entregas del proyecto

El proyecto se divide en dos entregas. La primera entrega corresponde a un prototipo funcional que debe incluir un agente orquestador inicial, un componente RAG funcional sobre los apuntes, una base vectorial, trazas de observabilidad, una interfaz mínima y un avance del informe.

La segunda entrega corresponde al sistema completo. En esta etapa se deben incorporar los agentes restantes, el servidor MCP, la base de datos transaccional ficticia, la memoria temporal e histórica, las reglas de seguridad, la observabilidad completa, la evaluación experimental y el informe final en formato IEEE.

III-J. Evaluación experimental

El sistema debe evaluarse mediante un conjunto de preguntas de prueba. Estas deben incluir preguntas factuales sobre los apuntes, preguntas de comparación entre temas, preguntas conversacionales de seguimiento, preguntas fuera del alcance de los apuntes, preguntas que requieran búsqueda web explícita y preguntas sobre transacciones ficticias.

Esta evaluación permite analizar el comportamiento del sistema, identificar errores y revisar si los agentes, herramientas, memoria y recuperación de información funcionan de manera adecuada.